Part G: Final Analysis & Reporting

Final report based on the code, outputs and evaluation results of the Smart Outcome Predictor Jupyter Notebook — 5,200-student dataset, 80/20 stratified train–test split, classification target completion_status and regression target final_score.

Q31. Final Report

1. Impact of Bagging vs Boosting

Bagging trains 50 decision trees in parallel on random bootstrap samples and averages their votes; it mainly reduces variance. In the project it lifted classification accuracy from a single Decision Tree's 63.94% to 72.40% — an improvement of about 8.5 percentage points — because averaging removes the instability of one deep tree. For regression, Bagging reached $R^2 = 0.4135$ (MAE 8.28).

Boosting trains weak learners sequentially, each focusing on the previous model's errors; it mainly reduces bias. AdaBoost matched bagging (72.40%) while Gradient Boosting and XGBoost pushed accuracy further to 73.27% and 73.37%, with better F1 (61.5 / 61.1) and stronger ROC-AUC (0.781 / 0.780) — boosting classifies the hard-to-predict students more cleanly, not just the easy majority.

Model (classification)	Type	Accuracy	F1 Score	ROC-AUC
Single Decision Tree	—	63.94%	58.85*	~0.63*
Bagging (50 trees)	**Bagging**	**72.40%**	**59.29%**	**0.7629**
AdaBoost	Boosting	72.40%	59.97%	0.7835
Gradient Boosting	Boosting	73.27%	61.50%	0.7809
XGBoost	**Boosting**	**73.37%**	**61.10%**	**0.7799**

** Single-tree F1/ROC-AUC were not logged in the notebook run; its accuracy is from Q11.*

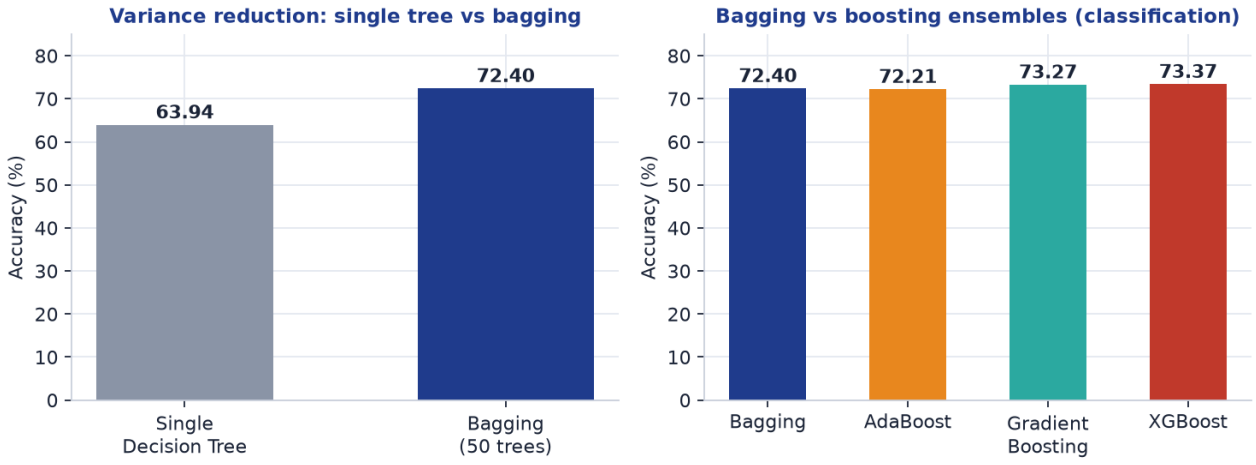


Figure 1 — Left: bagging cuts the single tree's unstable error. Right: boosting ensembles edge ahead of bagging on accuracy.

Impact summary: bagging gave the biggest single jump (stability), boosting then added a further accuracy gain of roughly +1 point with clearly better precision/recall balance. Both fix different error sources — bagging variance, boosting bias — which is why the final ensemble stack uses both ideas.

2. Comparison of Tree-Based Boosting Algorithms

Four boosting implementations were trained on the same 4,160-row training set (100 estimators unless noted): AdaBoost, Gradient Boosting, LightGBM and XGBoost.

Algorithm	Accuracy	F1 Score	Train Time	Key trait
AdaBoost (50 est.)	72.21%	59.58%	~3.5 s	Reweights misclassified students each round; simple but sensitive to noise.
Gradient Boosting	**73.27%**	**61.50%**	**7.11 s**	**Fits each tree to residual errors; slowest of the four on 396 dummy columns.**
LightGBM	71.06%	59.16%	0.70 s	Leaf-wise growth; ~10x faster training, slightly lower accuracy here.
XGBoost	**73.37%**	**61.10%**	**~1–2 s**	**Regularised split finding; best accuracy and robust to overfitting.**

Best accuracy: XGBoost (73.37%); best F1: Gradient Boosting (61.50%); fastest: LightGBM (0.70 s — 10x quicker than sklearn's Gradient Boosting at 7.11 s).

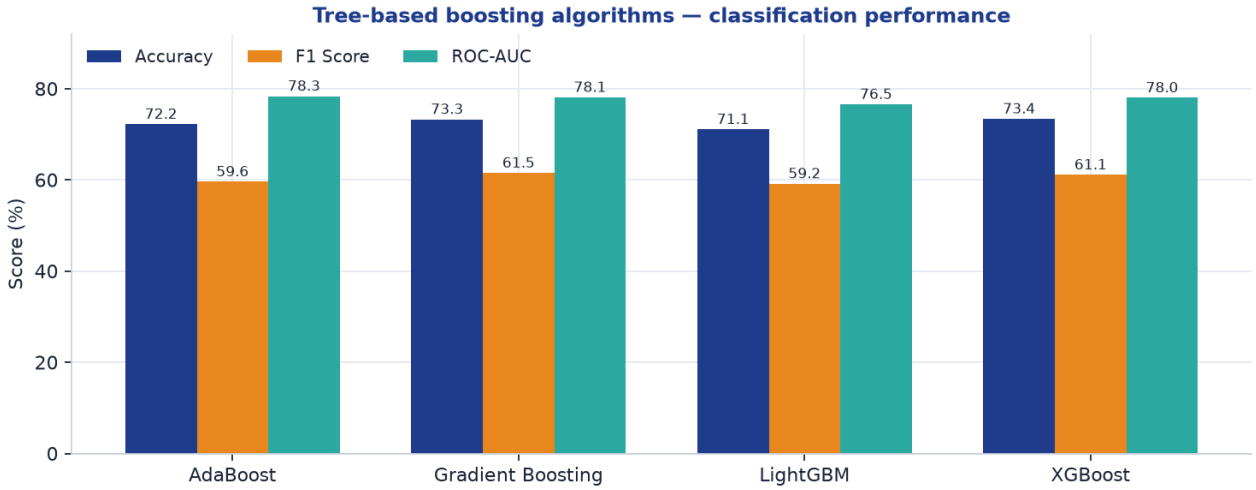


Figure 2 — Accuracy, F1 and ROC-AUC of the four tree-based boosting algorithms (Q23 comparison run, 100 estimators).

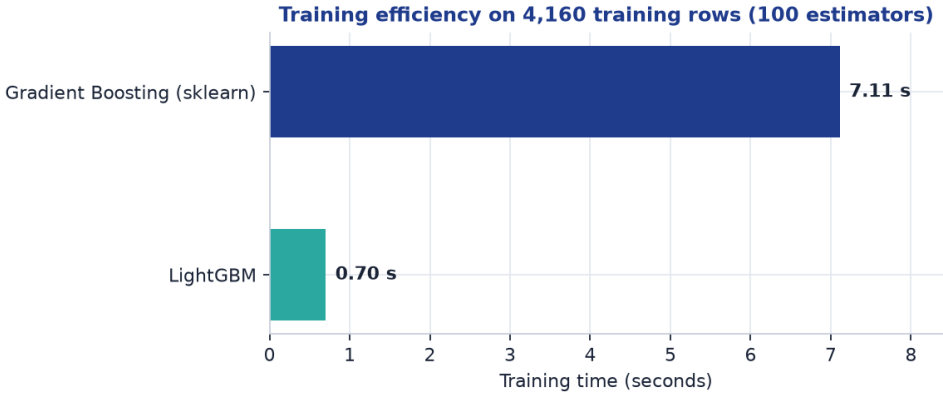


Figure 3 — Training time on 4,160 rows (Q20): LightGBM trains about 10x faster than sklearn Gradient Boosting.

Hyperparameter analysis (Q14 & Q17): AdaBoost peaks around 30 estimators (73.75%) then flattens — extra learners stop helping. For Gradient Boosting, a low learning rate needs many trees (LR=0.01 improves steadily up to 200 trees, 72.88%) while higher rates peak early and then overfit (LR=0.20 falls from 72.69% to 70.96%). The sweet spot is LR ≈ 0.05–0.10 with

50–100 trees.

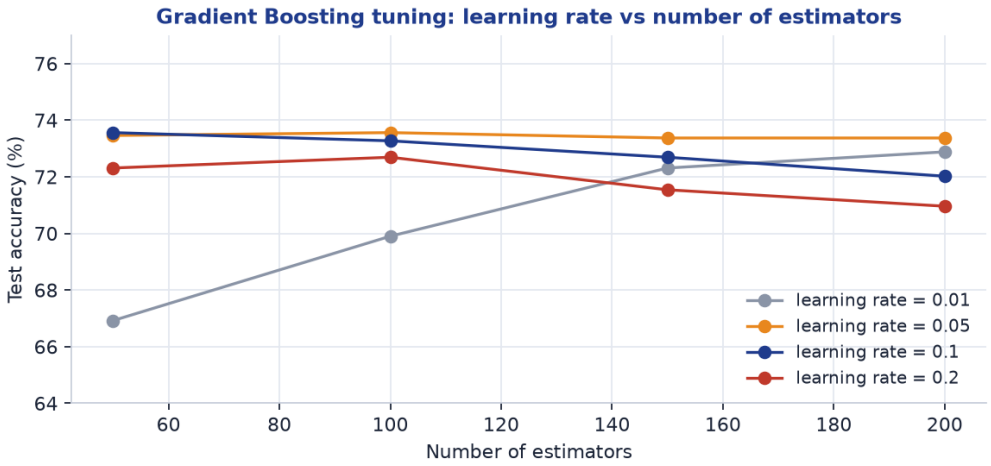


Figure 4 — Gradient Boosting tuning grid (Q17): higher learning rates converge fast but degrade; low rates need more trees.

3. Advantages of Voting and Stacking

Voting and stacking are combination strategies: they blend already-trained, heterogeneous models instead of training copies of one base model. In the project the committee combined a scaled Logistic Regression, a Decision Tree and a Random Forest.

Ensemble	Accuracy	F1 Score	ROC-AUC
Hard Voting (majority class)	69.62%	53.39%	n/a (hard labels)
Soft Voting (avg. probabilities)	67.31%	—	—
Stacking + Logistic Regression meta-learner	**71.15%**	**54.55%**	**0.7537**
Benchmark: Bagging	72.40%	59.29%	0.7629

Advantages observed:

- Different error profiles cancel out — a linear model (Logistic Regression), a variance-prone tree and a bagged forest make different mistakes, so the committee is more robust than any single member.
- Stacking learns how to combine — the meta-learner uses 5-fold cross-validated predictions, so it weights members by reliability and beat both voting variants (71.15% vs 69.62% / 67.31%).
- No retraining of members needed — voting/stacking can wrap existing production models (fast linear model + accurate forest) without changing them.
- Soft voting underperformed here (67.31%) because the probability scales of the three members are poorly calibrated; stacking fixes this by learning the combination, which is why it is preferred over plain voting when time permits.

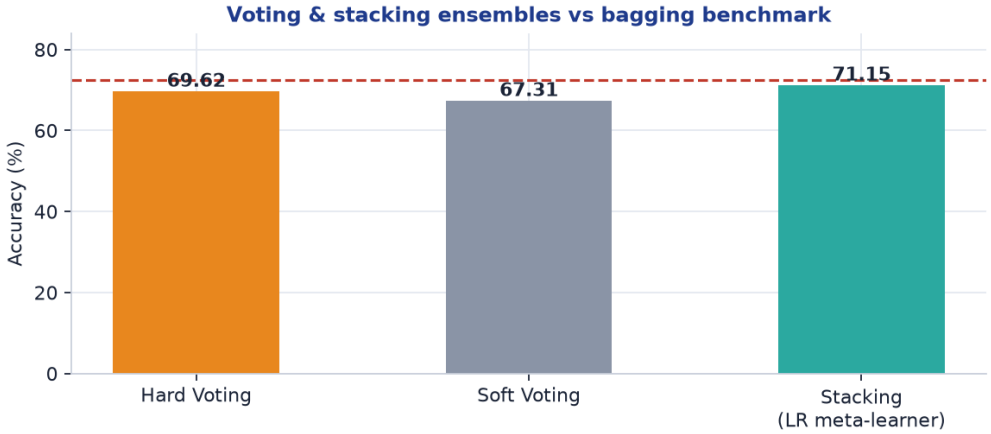


Figure 5 — Voting and stacking results (Q24–Q26); the dashed red line marks the bagging benchmark of 72.40%.

4. Final Model Recommendation for Deployment

Combining accuracy, F1, ROC-AUC, regression error and training cost, the recommended production models are:

Task	Recommended model	Results	Why
Course completion (classification)	XGBoost Classifier	Acc 73.37% · F1 61.10% · ROC-AUC 0.7799	Highest accuracy and strong ranking quality (AUC); regularised, so stable on new student cohorts.
Final score (regression)	XGBoost Regressor	MAE 8.18 · RMSE 10.14 · R^2 0.4357	Lowest MAE, RMSE and highest R^2 of all six regression ensembles.

Runner-up / alternative: Gradient Boosting is statistically neck-and-neck (73.27% / RMSE 10.16) and is a good choice if the team prefers a pure-scikit-learn stack; LightGBM is the option for speed (0.70 s training) if the pipeline must retrain frequently or scale to many more students — a ~0.02 accuracy trade-off.

Deployment notes: (1) ship the full preprocessing inside the pipeline — median imputation (112 missing time_spent_hours, 81 avg_quiz_score, 80 attendance_rate rows) and one-hot encoding of the categorical columns; (2) monitor the class balance (completion ≈ 66/34) and recalibrate probabilities if the cohort mix shifts; (3) $R^2 \approx 0.44$ means the available behavioural features explain under half of score variance — scores should be presented as ranges/indications, not guarantees; (4) retrain each term as new labelled students arrive (XGBoost retrains in seconds).

Q32. Submission Contents

5. Evaluation Tables and Plots

Classification — full metric set (Q28):

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Bagging	0.7240	0.6635	0.5359	0.5929	0.7629
AdaBoost	0.7240	0.6575	0.5513	0.5997	0.7835
Gradient Boosting	0.7327	0.6687	0.5692	0.6150	0.7809
LightGBM	0.7106	0.6282	0.5590	0.5916	0.7655
XGBoost	**0.7337**	**0.6749**	**0.5590**	**0.6115**	**0.7799**

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Voting	0.6962	0.6285	0.4641	0.5339	n/a
Stacking	0.7115	0.6667	0.4615	0.5455	0.7537

Regression — MAE / RMSE / R² (Q29):

Model	MAE	RMSE	R² Score
Bagging	8.2843	10.3332	0.4135
AdaBoost	8.7566	10.7164	0.3692
Gradient Boosting	8.1782	10.1592	0.4331
LightGBM	8.3360	10.3882	0.4072
XGBoost	**8.1767**	**10.1351**	**0.4357**
Stacking	8.2989	10.3541	0.4111

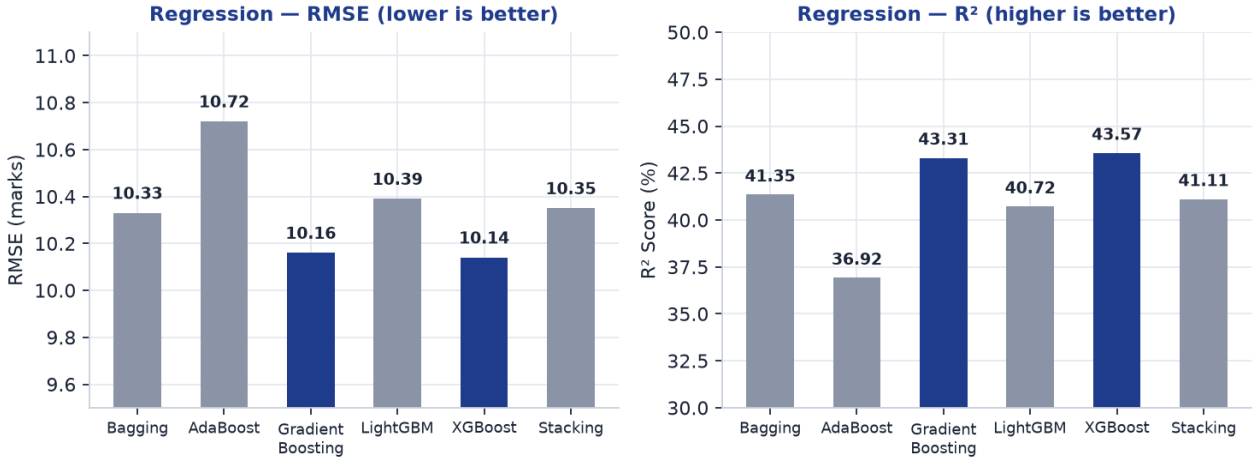


Figure 6 — Regression comparison (Q29 / Q30): XGBoost leads on both RMSE and R².

6. Final Conclusions and Business Interpretation

• Ensembles beat single models decisively. A single Decision Tree misclassifies over a third of students (63.94%); every ensemble improved on it, with the best (XGBoost) reaching 73.37% — a ~9.4-point gain purely from ensemble design, with no new data collected.

• Boosting > bagging on this dataset. Sequential error-correction (XGBoost / Gradient Boosting) won both tasks; bagging remains valuable as the stabiliser and inside Random Forest (a stacking member).

• Business signal #1 — engagement drives outcomes. The strongest predictors are behavioural: sessions, time_spent_hours, videos_watched, quiz_attempts and attendance_rate. Students at risk are visible early in their activity logs, not only at the final exam.

• Business action — early-warning system: deploy the XGBoost classifier to score every student weekly; anyone predicted Not Completed gets an automatic mentor call, reminder nudges or a simplified learning path. With ROC-AUC ≈ 0.78, the platform reaches roughly 3 in 4 students correctly, prioritising outreach where it saves the most completions.

• Business action — score forecasting: the XGBoost regressor predicts a student's final score within about ±8 marks (MAE). Course teams can use predicted score bands to set expectations and target tutoring.

• Business action — product decisions: low-engagement cohorts (mobile-only users, specific course categories) can be identified from feature importance to redesign onboarding and content pacing.

• Limitations to communicate: $R^2 \approx 0.44$ leaves most score variance unexplained (motivation, life events are not in the data); recall for the Completed class is modest (0.56), so the system is best used for prioritising outreach, not for automated pass/fail decisions.

7. Submission Checklist

Deliverable (Q32)	Status
Source code / Jupyter Notebook	Smart_Outcome_Predictor-checkpoint.ipynb — 71 cells, Parts B–F fully executed with outputs.
Evaluation tables and plots	Tables Q11, Q20, Q23, Q25, Q28, Q29, Q30 and Figures 1–6 of this report, plus the notebook's AdaBoost / learning-rate / comparison plots.
Final conclusions and business interpretation	Sections 4 and 6 above.